

Minecraft 建筑生成多智能体系统：从中文需求到可执行数据包

龙想 2300011196；石宇宸 2300011051

《大语言模型与信息决策》课程项目

2026 年 6 月

摘要

Minecraft 建筑生成任务同时涉及自然语言理解、空间规划、材料选择、几何约束、连通性校验和游戏数据包导出。若直接要求大语言模型输出具体方块坐标，系统容易出现非法材料、坐标不一致、房间不连通和命令不可执行等问题。本文介绍一套面向 Minecraft Java 的建筑生成多智能体系统。系统将大语言模型限制在高层语义决策层，使其输出风格、体块、材料、房间拓扑和设计意图等 JSON；随后由本地确定性几何引擎完成 CSG 体块生成、BSP 房间划分、A* 连通路径、室内装饰、质量校验、自动修复和数据包导出。项目从最初的单一建筑智能体逐步迭代为多智能体流水线，并逐步加入语义规划、本地几何、质量检查、可视化预览和候选择优机制。当前版本支持无 API key 的 mock 模式，可生成 blueprint.json、architect_datapack、preview.html 和 run_report.md；全量测试结果为 176 项通过、0 项失败。

关键词：大语言模型智能体；Minecraft；多智能体系统；CSG；BSP；A*；数据包

1 引言

Minecraft 的开放世界为生成式建筑提供了直观的实验场景。用户可以用一句中文描述建筑需求，例如“建一个欧式大房子”或“建造一个木屋别墅”，但系统真正需要解决的问题远不止文本生成。一个可用的建筑结果必须包含合理尺度、入口、房间、楼梯、屋顶、窗、材料、内饰、路径和最终可执行的 Minecraft 命令。

项目早期的设想是让智能体直接把需求变成建筑蓝图或命令。实际尝试后，我们发现这一路径不稳定：大模型擅长描述建筑意图，却不擅长长期保持三维坐标、材料注册名和拓扑约束的一致性。由此，项目的核心问题被重新定义为：如何让大模型负责语义与设计决策，同时让本地程序负责确定性几何落地和工程校验。

本文围绕这一修改后的问题定义展开。系统目标不是制作一个实时连服机器人，而是构建从中文需求到 Minecraft 可安装数据包的完整、可复现、可测试流水线。

2 系统总体设计

系统采用“LLM 语义智能体与本地确定性几何引擎”相结合的混合架构，如图 1 所示。大模型主要负责开放需求理解、风格构思和高层 JSON 决策；本地程序负责坐标、材料、连通性、命令体积和数据包导出等确定性任务。

这种边界划分是项目最重要的构思修改：我们没有让模型直接写坐标，而是让它提出建筑意图，再由程序把意图变成可检查的 Minecraft 方块网格。

总体架构

让 LLM 做语义决策，让本地程序做可验证落地

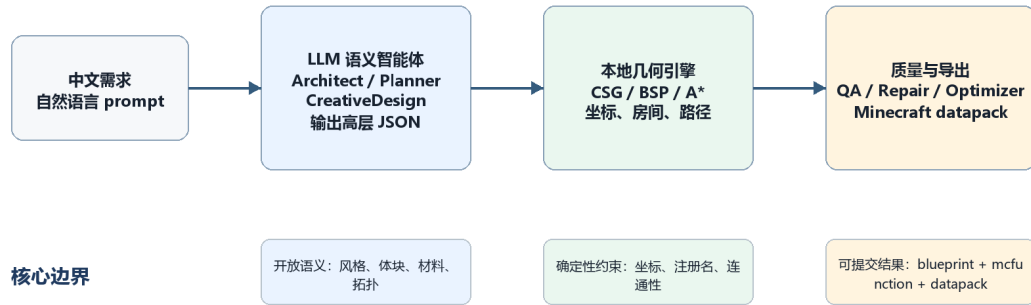


图 1 系统总体架构：LLM 负责语义决策，本地程序负责几何落地与导出

表 1 系统层次与模块职责

层次	代表模块	主要职责
语义层	ArchitectAgent, PlannerAgent	将中文需求转化为风格、材料、体块和房间拓扑 JSON
几何层	CSGBuilder, BSPPartitioner, AStarPathfinder	生成外壳、房间、门洞、楼梯和可达路径
质量层	QA, Repair, Optimizer	校验材料、连通性、入口、装饰和命令数量
导出层	Exporter	输出 blueprint.json、mcfunction 和可安装 datapack

3 方法

图 2 展示了主流程。流程分为语义决策、确定性几何、质量闭环和数据包导出四段；每段都有明确输入输出，因此可以单独测试和回退。

3.1 语义智能体分工

主流程位于 construction_method_v1。用户输入首先被解析为 buildSpec，随后 ArchitectAgent 生成外壳语义，MaterialPaletteAgent 基于 Minecraft Java 1.21.1 方块目录校验材料，PlannerAgent 生成房间节点与连通边，CreativeDesignAgent 决定体块变体、平面排序、屋顶、立面和场地策略。

后续智能体继续细化建筑表现。StructureAgent 给出支撑和荷载路径语义，FacadeAgent、RoofAgent、SiteLandscapeAgent 分别规划立面、屋顶和场地，InteriorDetailAgent 与 DecoratorAgent 负责房间级家具、灯光、植物和功能台面。整个过程中，LLM 的输出被约束为 JSON 语义对象，而不是直接写入坐标命令。

3.2 确定性几何落地

几何层是系统稳定性的关键。CSGBuilder 将主体、侧翼、门廊、塔楼等语义体块合并为稀疏 voxel 网格，并抽取外表面、楼板、屋顶和开窗。BSPPartitioner 根据楼层功能和房间权重划分室内空间；AStarPathfinder 根据拓扑关系生成门洞、楼梯和可通行路径。这样可以避免 LLM 直接坐标生成中的穿墙、断路和房间重叠问题。

导出层将最终网格转换为 Minecraft 数据包。玩家复制 architect_datapack 到世界 datapacks 目录后，执行 /reload 和 /function architect:run 即可触发 clear + build。系统同时输出 preview.html 和 run_report.md，便于不进入游戏时检查结构、流程和统计信息。

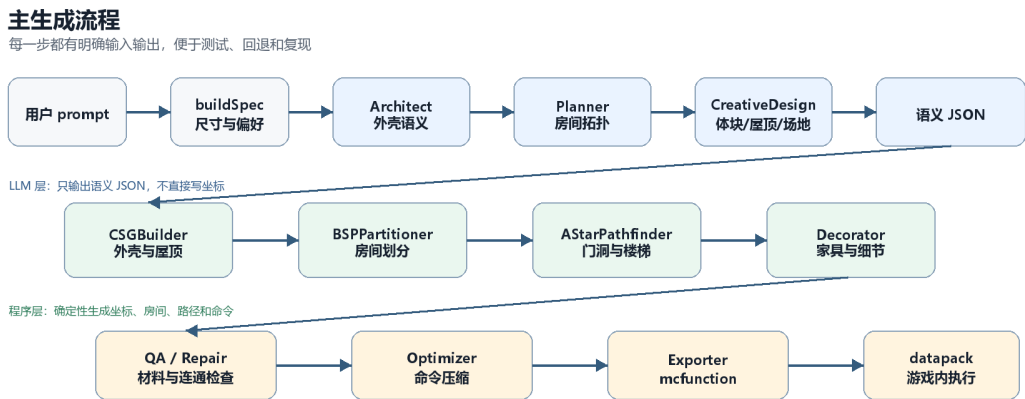


图 2 从中文 prompt 到 Minecraft datapack 的主流程

3.3 质量检查与可复现导出

在生成末端，系统不会直接信任任何单个智能体输出，而是把结构、材料、入口、房间可达性、装饰和命令数量放入统一检查。若发现缺入口、断路、非法方块或命令过多，Repair 与 Optimizer 会在导出前进行修复和压缩。

如图 3 所示，质量闭环的目标是让一次中文 prompt 最终落到可复现的工程产物：同一个 seed 可以复现同一份 blueprint、预览页、运行报告和 Minecraft 数据包。

质量检查与导出闭环

把一次生成结果变成可复现、可检查、可安装的数据包

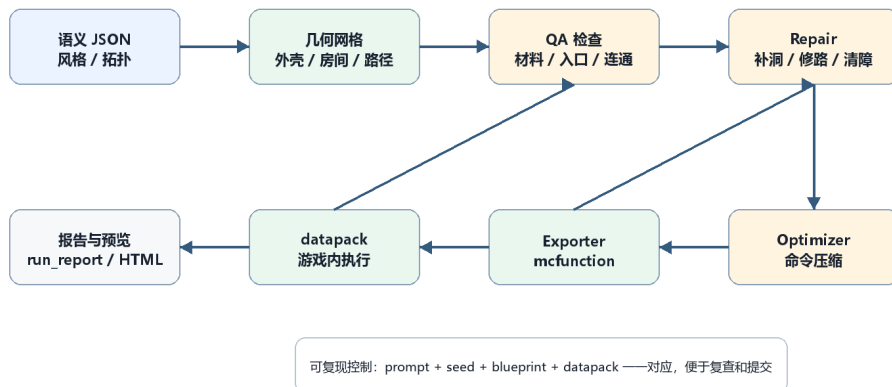


图 3 质量检查与可复现导出闭环

4 实现过程与构思迭代

项目的主要工作并不是一次性写出最终架构，而是在多轮实现中逐步修改问题定义。图 4 概括了 Git 历史中几个关键阶段：从初始 agent，到多智能体拆分、语义规划、本地几何、质量检查和提交版本整理。

这些变化反映了项目构思的转向。最初我们关注“如何生成一个房子”，后来逐步意识到课程项目更需要展示智能体系统如何分工、如何约束大模型、如何把开放语义落地为可靠工程产物。

构思与实现迭代

从“生成一栋房子”转向“可复现的智能体建筑流水线”

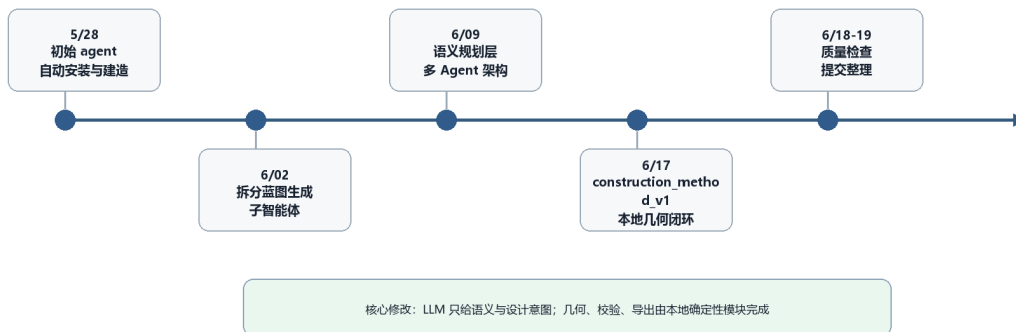


图 4 项目构思与实现迭代路线

5 实验与结果

当前全量测试为 176 项通过、0 项失败，其中包括课程提交文档测试；原项目功能基线为 173 项通过、0 项失败。测试覆盖 Architect fallback、Planner、CSG、BSP、A*、Decorator、候选择优、LLM provider、材料目录和可视化输出等模块。

本地样例 out/2026-06-19-145532212 使用 prompt“建造一个木屋别墅”。run_report.md 记录了 LLM 调用状态、seed、几何统计、装饰数量、QA 状态和 Minecraft 使用步骤。该样例说明系统并非只生成文本，而是在输出前经过结构、材料、连通和命令层面的检查。

为便于后续补充真实截图，仓库另整理了 docs/recommended-prompts.md，其中包含带固定 prompt-id 的推荐样例和截图取景建议。

表 2 本地样例运行结果

指标	结果	说明
建筑尺寸	43 x 16 x 57	来自 run_report.md 的几何校验结果
房间可达性	14/14	入口可达房间全部通过
QA 检查	9/9	结构、材料、入口和装饰检查通过
装饰数量	819	由室内与风格装饰模块写入
命令压缩	2692 -> 1251	压缩倍率约 2.15x，降低数据包函数体积
建造入口	/function architect:run	数据包内置清理与建造函数

6 讨论

从课程要求看，项目的整体性体现在所有模块都围绕同一条主线协作：中文需求输入、智能体语义决策、本地几何落地、质量检查和数据包导出。项目的创新性主要体现在三点。第一，系统清晰区分 LLM 与程序算法的职责，避免让大模型直接处理坐标和命令。第二，语义 JSON 与本地几何之间有清晰接口，便于单独测试和回退。第三，项目保留 mock 兜底和自动化测试，使没有 token 或现场网络时仍能复现完整流程。

项目也存在局限。当前版本不是 Mineflayer 实时机器人，也不模拟玩家逐块放置；GDMC HTTP 客户端虽然保留，但主交付仍是数据包导出。自动检查可以发现结构、材料和连通问题，却不能完全替代玩家视角下的尺度、镜头、路径和 block readability 检查。后续可补充真实游戏截图、人工评分样本，以及更严格的视觉检查。

7 结论

Minecraft Constructing Agents 构建了一条从中文建筑需求到 Minecraft 可执行数据包的完整流水线。项目的核心经验是：在开放生成任务中，大语言模型更适合作为语义和设计决策者，而不是直接坐标生成器；本地几何算法、材料目录、连通性校验和命令优化可以为其提供稳定落地能力。

通过多智能体分工、CSG/BSP/A* 几何引擎和质量闭环，项目将一个模糊的建房需求转化为可复现、可检查、可安装的工程产物。相比最终单次生成效果，项目更重要的贡献在于展示了一个课程级智能体系统如何从想法、失败尝试和架构调整中逐步形成。

AI 辅助说明

本项目合理使用 AI 工具辅助开发和文档整理。AI 辅助主要用于代码修改建议、文档初稿整理、报告润色、测试清单和网页排版检查；项目方向、系统边界、架构取舍、运行调试、测试执行、结果确认和最终整合由小组完成。报告和网站不伪造运行截图；若最终版本需要图像材料，应使用 preview.html 或 Minecraft 游戏内真实运行结果截图。

参考资料

- [1] Minecraft-Constructing-Agents 项目仓库与 README, <https://github.com/CityC196/Minecraft-Constructing-Agents>。
- [2] 本地样例运行报告: out/2026-06-19-145532212/run_report.md。
- [3] 课程提交展示页与提交清单: docs/index.html, SUBMISSION.md。
- [4] 推荐 Prompt 库与截图建议: docs/recommended-prompts.md。